Correlated Exploration for Cooperative MARL via a Gaussian Copula

Michał Słowakiewicz

Jakub Woźniak

Project 9 — Reinforcement Learning

Abstract

In cooperative multi-agent RL (MARL), agents must often *coordinate* their actions to receive any reward. Standard ε-greedy explores each agent independently, so the chance of stumbling on a coordinated joint action decays exponentially in the number of agents. We replace independent ε-greedy with **correlated ε-greedy**: the random action that exploring agents take is coupled through a Gaussian copula whose correlation matrix is the cosine similarity of per-agent features, so similar agents tend to pick the same action. On a coordination-bottlenecked toy task (*Hallway*) this yields a **+64–94%** relative early-learning gain; on sparse *Level-Based Foraging* it finds **2.6–2.9**$\times$ more coordinated successes; and on a GPU-scale StarCraft-like task (SMAX `2s3z`, JAX) it makes QMIX training markedly more reliable (mean final win $0.81 \to 0.91$ with $\sim 3\times$ lower seed variance; suggestive but not significant at $n{=}5$). The observation is the most reliable similarity source, coupling situationally-similar agents with no role information supplied.

1 Introduction

We study the decentralised partially-observable MDP (Dec-POMDP) under centralised training, decentralised execution (CTDE): each agent i observes o_i, picks a_i, and the team receives a single shared reward r.

The coordination problem. Consider a task that rewards the team only when all n agents simultaneously take a specific matching action. Under independent ε-greedy, the probability that all n explore the matching joint action in one step is $\propto \varepsilon^n$ — exponentially small. If agents instead explored in a *correlated* way, tending to take similar actions together, they would sample coordinated joint actions far more often.

Hypothesis. *Replacing independent ε-greedy with correlated exploration, where the correlation reflects how similar two agents currently are, accelerates learning on coordination-bottlenecked tasks.*

2 Method

We keep the ε-greedy template — with probability ε an agent acts randomly, else greedily — but *correlate which random action exploring agents take*. *Who* explores stays an independent Bernoulli (same exploration budget as the baseline); only the random action is coupled.

Gaussian copula. A copula imposes a desired correlation on uniform variables while keeping each marginal uniform. Given a correlation matrix R (PSD, unit diagonal): (i) factorise $R = LL^\top$; (ii) sample $z \sim \mathcal{N}(0, I)$ and set $y = Lz \sim \mathcal{N}(0, R)$; (iii) map $u_i = \Phi(y_i)$, giving uniform marginals *correlated* according to R. An exploring agent then takes the action $\lfloor u_i \, |A_i| \rfloor$ among its $|A_i|$ available actions. Because the u_i are correlated, agents with high R_{ij} pick the *same* action — a coordinated joint action (both stepping on switches, or focus-firing the same enemy). Independent ε-greedy is the special case $R = I$.

Correlation from cosine similarity. Let f_i be a feature vector for agent i and $\hat{f}_i = f_i / \|f_i\|$. Then

$$R_{ij} = \hat{f}_i^\top \hat{f}_j = \frac{f_i^\top f_j}{\|f_i\| \, \|f_j\|} = \cos\theta_{ij}, \quad R = \hat{F}\hat{F}^\top + \lambda I, \tag{1}$$

where $\hat{F}$ stacks the normalised features. R is a Gram matrix, hence positive semi-definite ($x^\top R x = \|\hat{F}^\top x\|^2 \geq 0$) with unit diagonal — a valid correlation matrix; the jitter $\lambda = 10^{-3}$ keeps the Cholesky well-conditioned. We compare three similarity sources f_i: the (augmented) observation `obs`, the Q-value vector `q_values`, and the Q-network backbone activation `hidden`. The design is role-agnostic: nothing tells the method which agents share a role; similar agents simply get correlated.

3 Experimental Setup

Environments. *Hallway* [3]: two agents on separate corridors (lengths 3, 4), each seeing only its own position; the team is rewarded only when both reach position 0 in the same step. *Level-Based Foraging* [6] (`Foraging-5x5-2p-1f-coop`): two agents must load one food item *together*; sparse reward. *SMAX `2s3z`* [4]: a JAX StarCraft-like battle, 2 stalkers + 3 zealots vs. a mirrored enemy team — a 5-agent, two-role task.

Algorithms. Two value-decomposition backbones with parameter sharing (one shared Q-net; agents disambiguated by a one-hot id): VDN [1] ($Q_{\text{tot}} = \sum_i Q_i$) and QMIX [2] (monotonic mixer with a hypernetwork over the global state). DQN-style training, 5 seeds per configuration. On

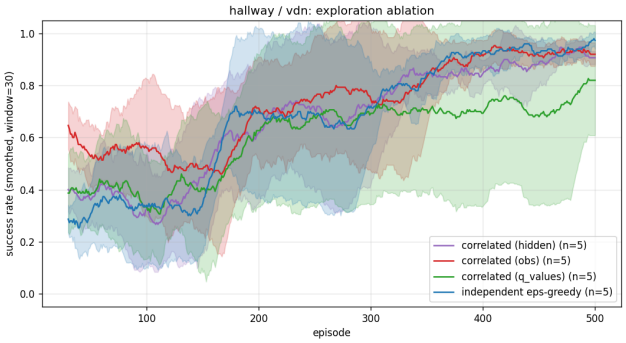

Figure 1: *Hallway* (VDN). Correlated-`obs` learns markedly faster early; all variants converge, so the gain is a speed-up.

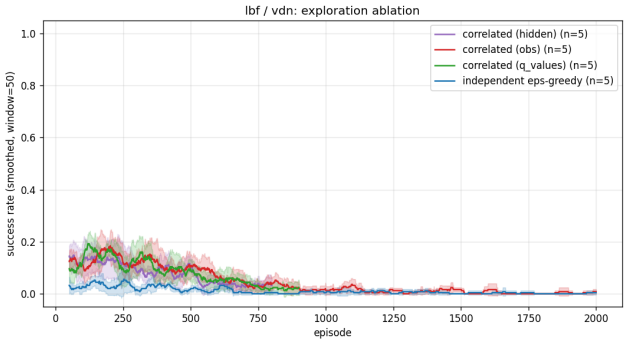

Figure 2: *LBF* (VDN). All methods collapse (catastrophic forgetting), but correlated reaches a 2.6–2.9× higher *peak*.

Table 1: Hallway success rate at episodes 50–100.

Backbone	Independent	Correlated-`obs`	Relative
VDN	0.348	**0.572**	+64%
QMIX	0.276	**0.536**	+94%

LBF we oversample the rare successful transitions (balanced replay).

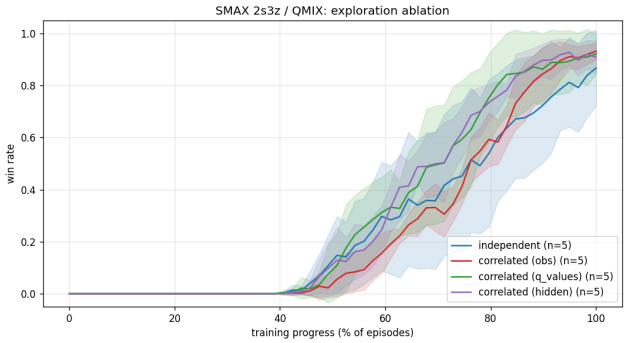

Figure 3: SMAX `2s3z` (QMIX). Correlated variants reach a higher final win rate than independent (0.91 vs. 0.81) and get there sooner.

4 Results: Toy Benchmarks

Hallway: faster early learning. Correlated exploration with the `obs` source learns markedly faster in the early phase, when exploration matters most (Fig. 1, Table 1). All variants eventually reach a high success rate, so the benefit is a **speed-up**, exactly as the hypothesis predicts for a coordination-bottlenecked task.

LBF: the mechanism works even where learning fails. LBF is dominated by catastrophic forgetting — every method peaks early then collapses toward zero (Fig. 2). The *peak* success rate, however, cleanly separates the methods: independent peaks at 0.07–0.09, while the best correlated variant peaks at 0.20–0.24 — **2.6–2.9× higher**. The exploration mechanism finds the coordinated successes independent exploration misses; failing to *retain* them is a separate problem of value learning under extreme sparsity.

Which similarity source? `obs` is the most reliable: it gives the largest, most consistent early gain and the highest LBF peak. `q_values` and `hidden` are inconsistent — early in training the network is random, so its Q-values and activations carry little reliable signal, whereas the observation is informative from step one.

5 Results: SMAX `2s3z` (JAX, GPU)

We reimplemented the method in pure JAX (Flax/Optax), vectorising 128 parallel environments with `jax.vmap` (∼6900 env-steps/s on one RTX 2080 Ti).

Stabilising a from-scratch learner. Our first runs did not learn: the TD loss *diverged* to 10^4–10^6 (Q-value explosion, not under-training). Four standard fixes resolved it — Double DQN [5] targets, Polyak soft target updates (τ=0.005), lr $= 10^{-4}$, and a `LayerNorm` on the QMIX mixer's unnormalised global-state input (values up to ∼24 otherwise blew up the abs()-weighted hypernetwork). The best configurations then reach a 95–100% win rate.

Correlated exploration helps most on the harder mixer. Under VDN (simple additive mixer) every variant converges to ∼0.95; correlated only reaches the 50%-win milestone ∼6% sooner. Under QMIX (harder monotonic mixer) the effect is larger but must be read carefully (Table 2, Fig. 3): independent has a much higher mean *and* a much larger seed variance (0.81 ± 0.19; one seed collapses to 0.48), while all correlated variants are higher and far more reliable (0.90–0.91, std ≤ 0.12). A bootstrap over the 5 seeds gives a mean gap of $+0.10$ for the best correlated variant with 95% CI $[-0.03, 0.28]$ and $P(\text{correlated} > \text{independent}) \approx 0.91$ — *suggestive but not significant at* n=5. The robust, reproducible effect is therefore the **reduced variance / improved reliability** of correlated exploration (it avoids the failed runs that independent suffers) plus the consistent early speed-up; a firm claim on the mean final-win gain would need more seeds.

When does the correlation encode roles? The copula recomputes R live, and exploration peaks early (high ε), so

Table 2: SMAX `2s3z`: final win rate (mean ± std over 5 seeds).

	Indep.	corr-`obs`	corr-`q_values`	corr-`hidden`
VDN	0.95±.02	0.95±.01	0.94±.01	0.95±.01
QMIX	0.81±.19	0.91±.06	0.90±.12	0.91±.07

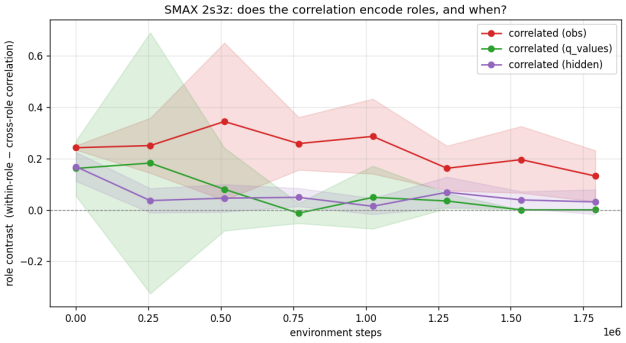

Figure 4: Role contrast (within-role − cross-role correlation) over training. `obs` stays positive through the exploration-heavy phase; `q_values` is noisy and `hidden` weak.

when R carries role structure matters. We summarise each snapshot by a role-contrast score (within-role minus cross-role correlation; Fig. 4). `obs` keeps a clearly positive contrast through the exploration-heavy phase — the observation always encodes unit type, independent of the (initially random) network. `q_values` is noisy and `hidden` weak. By convergence `q_values` is essentially uniform (∼0.99): a converged policy has large, all-positive Q-vectors, and cosine between two large positive vectors is dominated by their shared offset (mean-centring drops it to ∼0.85), so `q_values` cannot discriminate agents. The `obs` coupling is largely aligned with unit type — the two stalkers are consistently the most-correlated pair (∼0.9) — but reflects *situational* similarity (positions, visible enemies), not a clean role partition: which zealot couples with the stalkers varies by seed. Coupling situationally-similar agents is arguably the right behaviour.

6 Discussion and Conclusion

Correlated exploration helps where (a) coordination is the bottleneck and (b) learning is stable enough to exploit discovered successes (Hallway, SMAX-QMIX). Where learning itself is unstable (LBF), it still finds more successes but the gain is not retained. `obs` is the safe default similarity source.

Limitations. (i) The *Hallway* observation is a single non-negative scalar, so its cosine matrix is ≈ all-ones regardless of state; there correlated-`obs` reduces to "explore the same action." That validates *that* correlated exploration helps, but not the discriminative value of the cosine similarity — which is genuinely exercised only by the higher-dimensional LBF and SMAX observations. (ii) The SMAX final-win improvement is suggestive but not significant at $n{=}5$ (Sec. 5); the robust claim is the reduced variance and faster learning. (iii) We correlate actions only *within a single step*, unlike trajectory-level committed exploration (e.g. MAVEN); per-step coupling already raises the rate of coordinated joint actions, but temporally-extended coupling is a natural extension. (iv) R is recomputed greedily each step rather than annealed; only 2 agents in the toy tasks.

Overall, coupling agents' exploratory *actions* through a Gaussian copula parameterised by cosine similarity is a simple, drop-in change that measurably accelerates cooperative MARL on coordination-bottlenecked tasks and improves exploration quality even where value learning fails — scaling from toy grids to a GPU-scale StarCraft-like task.

Use of AI Tools

During the development of this project, **Claude Code** (Anthropic Claude Opus) was used as a programming assistant: implementing substantial parts of the PyTorch toy-environment stack and the JAX/JaxMARL SMAX pipeline, assisting with cluster debugging (e.g., resolving CUDA/cuDNN compatibility and multi-process GPU OOM issues), generating figures, and helping draft this report. *Worked well:* the tool was effective for rapid prototyping and for diagnosing specific implementation bottlenecks — notably the SMAX Q-divergence and the ∼50× `vmap` speed-up. *Did not:* the assistant occasionally produced conceptually flawed logic. Most notably, its initial SMAX exploration strategy correlated *whether* each agent explored rather than *which action* it took — a critical conceptual bug that surfaced only under manual code review. Ultimately, while the tool accelerates implementation, human oversight and conceptual verification remained necessary throughout.

References

[1] P. Sunehag et al. Value-Decomposition Networks for Cooperative Multi-Agent Learning. *AAMAS*, 2018.

[2] T. Rashid et al. QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent RL. *ICML*, 2018.

[3] A. Mahajan et al. MAVEN: Multi-Agent Variational Exploration. *NeurIPS*, 2019.

[4] A. Rutherford et al. JaxMARL: Multi-Agent RL Environments and Algorithms in JAX. 2023.

[5] H. van Hasselt, A. Guez, D. Silver. Deep RL with Double Q-learning. *AAAI*, 2016.

[6] G. Papoudakis et al. Benchmarking Multi-Agent Deep RL Algorithms in Cooperative Tasks. *NeurIPS Datasets & Benchmarks*, 2021.